

Arquitectura de una Plataforma Inteligente de Triage Médico Modular Basada en IA, RAG y Arquitecturas Cloud

Natalia Espitia Espinel
Jesus Alberto Jauregui Conde
Mayerlly Suarez Correa

Programa de Ingeniería de Sistemas
Arquitectura Empresarial y Transformación Digital
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia

Resumen

Los sistemas de salud de América Latina siguen enfrentando barreras estructurales de acceso, saturación en atención primaria y baja disponibilidad de especialistas en zonas apartadas. Como continuación del estado del arte desarrollado en la primera entrega, este trabajo propone la arquitectura de una plataforma inteligente de triaje médico general asistida por Inteligencia Artificial. La solución integra una aplicación web para pacientes, un panel profesional, un backend modular basado en FastAPI y un pipeline de Retrieval-Augmented Generation (RAG) orientado a trazabilidad clínica. Se plantea una arquitectura objetivo sobre Amazon Web Services y un prototipo MVP local con autenticación por roles, flujo funcional de consulta, recuperación de evidencia documental y escalamiento hacia un profesional cuando el riesgo lo exige. El aporte principal del artículo consiste en traducir la evidencia del estado del arte en una propuesta técnica defendible, acompañada de un prototipo académico observable y de una validación funcional inicial sobre cobertura de requisitos y atributos de calidad.

Palabras clave: Salud digital, Triage inteligente, Arquitectura empresarial, FastAPI, React, RAG, AWS, HL7 FHIR, Trazabilidad clínica

1. Introducción y continuidad

La primera entrega del proyecto presentó un estado del arte sobre plataformas inteligentes de consulta médica modular basadas en IA y arquitecturas cloud, identificando tres hallazgos centrales: i) la necesidad de escalar soluciones de orientación clínica primaria en contextos de saturación sanitaria, ii) la relevancia de arquitecturas distribuidas e interoperables para sostener ese crecimiento, y iii) el papel de las técnicas de Retrieval-Augmented Generation (RAG) para reducir alucinaciones y aumentar la trazabilidad de las recomendaciones generadas por modelos de lenguaje [1, 2, 3, 4].

Actualmente se avanza desde la descripción del panorama tecnológico hacia una propuesta de solución concreta. El problema ya no es solo identificar qué tecnologías son prometedoras, sino demostrar cómo articularlas en una arquitectura coherente, defendible y compatible con el alcance académico del curso. En ese sentido, el trabajo se concentra en una plataforma de triaje médico general que permite registrar síntomas, estructurar el caso clínico, recuperar evidencia relevante y entregar una orientación con decisión explícita de autocuidado o escalamiento profesional.

La contribución principal del artículo es doble. Por una parte, propone una arquitectura objetivo trazable y modular para una plataforma de triaje asistido por IA. Por otra, delimita un prototipo MVP local que materializa el flujo de mayor valor del sistema y permite observar la relación entre arquitectura, contratos, componentes y decisiones de diseño.

1.1. Del estado del arte a la arquitectura propuesta

La literatura revisada muestra que el valor de los sistemas de triaje digital no depende únicamente de la precisión algorítmica, sino también de su capacidad para justificar recomendaciones, integrarse con procesos asistenciales y sostener crecimiento operativo [1, 5, 6]. En particular, los trabajos sobre RAG en salud resaltan la necesidad de reducir la dependencia de respuestas generadas sin sustento documental, mientras que los marcos de arquitectura digital en salud subrayan la importancia de interoperabilidad, seguridad y evolución controlada [2, 3, 7].

Sin embargo, entre esos antecedentes persiste un vacío práctico: muchas discusiones se quedan en beneficios generales de la IA o en experimentos acotados, pero no aterrizan con suficiente detalle una arquitectura defendible que conecte captura de síntomas, reglas de severidad, evidencia recuperada,

trazabilidad y escalamiento profesional. Este trabajo responde a ese vacío proponiendo una arquitectura en la que la **precisión clínica y la trazabilidad** operan como atributo de calidad dominante. Bajo este criterio, cada recomendación debe estar asociada a evidencias recuperadas, severidad estimada y una decisión explícita de autocuidado o escalamiento.

2. Objetivos y alcance

2.1. Objetivo general

Diseñar la arquitectura de una plataforma modular de triaje médico asistido por IA que combine aplicaciones web, backend orientado a servicios, almacenamiento operativo y vectorial, y un pipeline RAG orientado a recomendaciones clínicas trazables.

2.2. Objetivos específicos

- Definir la arquitectura objetivo de la solución en nube, identificando actores, contenedores, componentes y responsabilidades.
- Delimitar un prototipo MVP académico que cubra el flujo de mayor valor clínico y permita evolucionar hacia una implementación distribuida.
- Especificar entidades, estados, endpoints y decisiones de diseño necesarias para la siguiente fase de implementación.
- Realizar una validación funcional inicial del prototipo, diferenciando con claridad el diseño actual de la futura evaluación experimental.

2.3. Alcance del MVP

El MVP se restringe al flujo registro de síntomas, análisis/triage, recomendación fundamentada y escalamiento a profesional. En consecuencia, el sistema debe permitir autenticación básica por rol, creación de una consulta clínica, ejecución de un flujo de triaje y recomendación, recuperación de evidencia clínica desde una base de conocimiento curada y visualización de casos escalados por parte del profesional.

Quedan fuera de alcance en esta fase la agenda médica, la historia clínica real, la facturación, el monitoreo productivo, el despliegue cloud real y la validación cuantitativa formal del sistema.

3. Requisitos del sistema

3.1. Requisitos funcionales

La Tabla 1 resume los requisitos funcionales priorizados para la entrega.

Cuadro 1: Requisitos funcionales del MVP

ID	Descripción
RF-01	El sistema debe autenticar usuarios con rol de paciente o profesional.
RF-02	El paciente debe poder registrar síntomas, antecedentes básicos y contexto de la consulta.
RF-03	El backend debe transformar la entrada libre en un caso clínico estructurado.
RF-04	El sistema debe ejecutar un proceso de triage que estime severidad y tipo de acción recomendada.
RF-05	El sistema debe recuperar evidencia clínica desde una base documental curada antes de generar la recomendación.
RF-06	La recomendación debe mostrar texto orientativo, severidad estimada, fuentes utilizadas y decisión de escalamiento o autocuidado.
RF-07	Los casos con severidad alta o incertidumbre relevante deben escalar al panel profesional.
RF-08	El profesional debe visualizar casos escalados, evidencia consultada y salida generada por el sistema.

3.2. Requisitos no funcionales

Los requisitos no funcionales se organizan alrededor del atributo dominante de *precisión clínica y trazabilidad*:

- **RNF-01 Trazabilidad:** toda salida debe registrar fuentes, versión del prompt, fecha, severidad y decisión tomada.
- **RNF-02 Explicabilidad:** la recomendación debe distinguir entre evidencia recuperada y conclusión generada.
- **RNF-03 Modularidad:** los módulos de autenticación, intake, triage y RAG deben evolucionar de forma desacoplada.
- **RNF-04 Seguridad básica:** el prototipo debe separar roles y evitar exposición innecesaria de datos sensibles.
- **RNF-05 Extensibilidad:** la arquitectura debe permitir agregar dominios clínicos, canales y adaptadores de interoperabilidad.
- **RNF-06 Preparación cloud:** la solución local del MVP debe mapearse con claridad a una arquitectura futura en AWS.

4. Arquitectura general de referencia

4.1. Actores y contexto

La solución propuesta se organiza en torno a dos actores primarios: el paciente, que inicia la consulta y recibe una orientación, y el profesional de salud, que atiende casos escalados o revisa recomendaciones de mayor riesgo. Adicionalmente, la plataforma interactúa con un servicio de LLM y embeddings, una base de conocimiento clínica y un adaptador futuro hacia sistemas clínicos externos.

La Tabla 2 resume dichas vistas y muestra que la propuesta no se limita a una descripción abstracta, sino que conserva trazabilidad entre contexto, contenedores, componentes, flujos y despliegue.

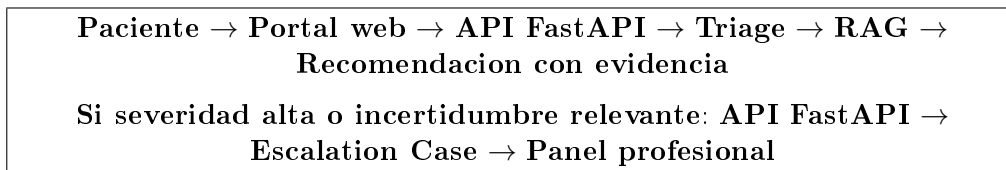


Figura 1: Esquema sintético del flujo principal del MVP.

Cuadro 2: Vistas arquitectonicas entregadas

Archivo	Tipo de vista	Proposito
01-c4-context.mmd	Contexto	Mostrar actores, plataforma, LLM, base documental y adaptador FHIR futuro.
02-c4-container.mmd	Contenedores	Delimitar frontends, API, servicios de triage/RAG y persistencia.
03-backend-components.mmd	Componentes	Detallar modulos internos del backend del MVP.
04-sequence-triage.mmd	Secuencia	Describir el flujo principal desde sintomas hasta recomendacion.
05-sequence-escalation.mmd	Secuencia	Describir el flujo de escalamiento hacia el profesional.
06-aws-deployment.mmd	Despliegue	Mapear la solucion objetivo sobre servicios de AWS.
07-data-model.mmd	Modelo de datos	Representar entidades y relaciones del MVP.

4.2. Contenedores principales

La arquitectura general se compone de un frontend para paciente, un frontend profesional, un backend FastAPI, un módulo de intake clínico, un módulo de triage, un módulo RAG, un módulo de trazabilidad, persistencia operativa y persistencia vectorial. La Figura 1 ayuda a observar que dichos contenedores no operan como piezas aisladas, sino como una cadena de valor cuyo producto final es una recomendación fundamentada y, cuando aplica, un caso listo para revisión profesional.

4.3. Arquitectura cloud objetivo

La proyección en nube privilegia la separación entre canales de atención, servicios de negocio y capacidades de IA. La solución objetivo considera hosting web con S3 y CloudFront, una puerta de entrada API, ejecución de servicios contenedorizados en ECS Fargate, almacenamiento relacional en Amazon RDS, soporte vectorial mediante OpenSearch o extensiones vectoriales sobre

PostgreSQL y un servicio de modelos equivalente a Amazon Bedrock para embeddings y generación.

La arquitectura cloud establece desde ahora un mapa de evolución coherente con el curso: crecimiento horizontal del backend, separación entre almacenamiento operativo y semántico, y observabilidad centralizada con servicios equivalentes a CloudWatch.

4.4. Trazabilidad clínica como criterio arquitectónico

El criterio dominante de diseño es la trazabilidad. En consecuencia, la arquitectura no permite que la recomendación dependa exclusivamente de una salida generativa. Antes de responder, el sistema debe recuperar documentos relevantes, asociarlos a la consulta y conservar un rastro de qué evidencia utilizó. Esta estrategia responde tanto al análisis de RAG en salud [2] como a la necesidad de supervisión y transparencia señalada por el marco ético colombiano para IA [7]. Frente a enfoques basados solo en generación libre, la propuesta busca reducir opacidad y facilitar auditoría clínica posterior.

5. Arquitectura del prototipo MVP

5.1. Recorte funcional

El prototipo implementa localmente el flujo de mayor valor académico y técnico: el paciente inicia sesión, registra síntomas y contexto básico, el backend crea una consulta estructurada, el módulo de triage determina severidad preliminar, el pipeline RAG recupera evidencia y genera una recomendación, y el caso se escala al profesional cuando el riesgo supera el umbral definido.

Esta delimitación evita inflar el prototipo con capacidades accesorias que no aportan valor a la defensa de la arquitectura, como agenda médica o integraciones hospitalarias reales. Al mismo tiempo, preserva un recorrido completo que permite validar la coherencia entre frontends, API, reglas de severidad, evidencia recuperada y salida trazable.

5.2. Diferencias entre arquitectura objetivo y MVP

La Tabla 3 muestra el recorte deliberado entre la arquitectura general y el prototipo.

Cuadro 3: Diferencias entre arquitectura objetivo y prototipo

Dimension	Arquitectura objetivo	MVP local
Canales	Frontends web y posibilidad futura de WhatsApp/SMS	Solo web para paciente y profesional
Persistencia vectorial	OpenSearch o PostgreSQL vectorial gestionado	PostgreSQL con <code>pgvector</code> planificado
IA generativa	Servicio gestionado en nube con seguridad y observabilidad	Abstraccion de servicio RAG lista para integrar modelo externo
Interoperabilidad	Adaptadores FHIR y conectores a sistemas clinicos	Capa conceptual no implementada
Operacion	Despliegue distribuido con monitoreo y logs centralizados	Ejecucion local, sin operacion productiva

5.3. Componentes del backend del MVP

Internamente, el backend se separa en `auth`, `consultations`, `triage`, `rag` y `professional_cases`. Esta organización hace visible una primera decisión de modularidad: autenticación, reglas de severidad, recuperación documental y lectura de casos escalados no comparten una única pieza monolítica, sino responsabilidades diferenciadas que luego podrán migrar hacia una arquitectura más distribuida.

5.4. Implementación observable en el repositorio

El repositorio ya materializa elementos concretos del MVP. En backend, `app/main.py` expone la API y registra rutas para autenticación, consultas y casos profesionales; `consultations.py` implementa la creación de consultas y la ejecución del triage; `triage_engine.py` calcula severidad a partir de intensidad y palabras clave de alerta; y `rag_service.py` carga el corpus curado desde `knowledge-base/clinical-guidelines` y recupera hasta tres

Arquitectura objetivo: canales web + servicios cloud gestionados + persistencia operativa y vectorial + adaptador FHIR. MVP actual: frontends React + API FastAPI + reglas de triage + RAG documental + almacenamiento en memoria y corpus curado local.
--

Figura 2: Relacion entre la arquitectura objetivo y el recorte funcional del MVP. La figura resume la diferencia entre la solucion pensada para evolucion futura y la implementacion academica observable en el repositorio.

fuentes relevantes para componer la recomendación. En frontend, el portal del paciente representa la captura estructurada de síntomas y la vista previa de una recomendación, mientras que el espacio profesional muestra la cola de casos escalados y la evidencia asociada.

Esta evidencia no convierte todavía al sistema en un producto listo para producción, pero sí demuestra correspondencia entre la arquitectura propuesta y una implementación académica verificable. Esa correspondencia es importante porque evita que el artículo prometa una solución puramente conceptual sin huella técnica en el repositorio.

6. Decisiones de diseño

La Tabla 4 resume las decisiones de arquitectura más relevantes adoptadas para la entrega.

7. Modelo de datos y contratos

7.1. Entidades principales

El modelo de datos del MVP se centra en ocho entidades: *User*, *Patient-Profile*, *Consultation*, *SymptomEntry*, *TriageResult*, *Recommendation*, *EvidenceSource* y *EscalationCase*. En conjunto, estas entidades permiten representar captura de síntomas, procesamiento, recomendación y transferencia del caso hacia el profesional.

Los estados de la consulta quedan definidos como submitted, triaged, recommended, escalated y reviewed. Esta secuencia no solo describe un flujo técnico; también ayuda a estructurar la trazabilidad del proceso y a preparar futuras mediciones de tiempos, volumen de escalamiento y seguimiento

profesional.

7.2. Contratos mínimos de API

La Tabla 5 resume los endpoints minimos del MVP.

7.3. Flujo principal

El comportamiento esperado del MVP es el siguiente: el paciente registra síntomas y contexto basal, el backend normaliza el caso clínico, el módulo de triage asigna una severidad preliminar, el módulo RAG recupera evidencia relevante del corpus curado, el sistema genera una recomendación fundamentada y, cuando corresponde, expone el caso al profesional con la información necesaria para su revisión. La Figura 1 sintetiza este encadenamiento y evita que el lector vea los componentes como una lista desconectada.

8. Resultados y validación inicial

8.1. Alcance de la validacion

El prototipo sí permite una validación funcional inicial, entendida como verificación de que el flujo principal, los contratos y los atributos de calidad priorizados tienen manifestaciones concretas en el repositorio. Esta validación se apoya en tres fuentes: el artículo, los artefactos de arquitectura y el código base del MVP.

8.2. Cobertura funcional observada

La Tabla 6 resume la cobertura funcional observable en el repositorio y sirve como evidencia minima de implementacion.

8.3. Evaluacion de atributos de calidad

La Tabla 7 presenta una evaluacion cualitativa de atributos de calidad relevantes para esta fase. La intencion no es sobrestimar el prototipo, sino mostrar que el diseño priorizado si deja rastros verificables en la implementacion actual.

8.4. Discusion frente al estado del arte

Los resultados deben interpretarse como evidencia de coherencia arquitectonica y no como prueba de desempeno clinico. Aun asi, muestran un avance relevante frente a enfoques que delegan la orientacion clinica a generacion libre sin explicar sus fuentes. En el prototipo propuesto, la recomendacion se acompaña de evidencia recuperada y de una decision de severidad, lo que alinea la implementacion con la preocupacion por trazabilidad senalada en la literatura sobre RAG en salud [2]. Del mismo modo, el escalamiento explicito hacia el profesional responde a la necesidad de mantener supervision humana en casos de mayor riesgo o incertidumbre [7].

La principal debilidad de esta validacion es que aun no incorpora metricas cuantitativas de tiempo de respuesta, precision de clasificacion o usabilidad observada. Esa ausencia limita cualquier comparacion dura con trabajos previos y obliga a presentar los resultados con prudencia. Sin embargo, la evidencia reunida si permite sostener que existe correspondencia entre problema, arquitectura y prototipo.

9. Riesgos, límites y siguiente fase

9.1. Riesgos

Los riesgos principales identificados son la falsa precisión, la cobertura documental insuficiente, la ambigüedad clínica y la dependencia futura del proveedor de IA. Estos riesgos no invalidan la propuesta, pero sí condicionan la forma en que debe evolucionar: más evidencia documental, mejor validación del triage, observabilidad y criterios más finos de escalamiento.

9.2. Límites de la entrega

Esta entrega no pretende demostrar desempeño clínico real, ni sustituir juicio médico, ni desplegar la solución en un entorno productivo. Su finalidad es fijar una arquitectura defendible, coherente con el estado del arte y suficientemente detallada para guiar la implementación posterior. En consecuencia, los resultados reportados en la sección anterior deben leerse como validación funcional inicial y no como evaluación experimental definitiva.

9.3. Trabajo futuro

Este debe concentrarse en implementación robusta del pipeline RAG, evaluación funcional y cuantitativa del prototipo, pruebas de atributos de calidad relevantes, integración de observabilidad y endurecimiento de seguridad, y demostración distribuida con despliegue en nube si el tiempo lo permite.

10. Conclusiones

La arquitectura propuesta convierte los hallazgos del estado del arte en una solución técnica concreta para triaje médico general asistido por IA. El trabajo no se limita a enumerar tecnologías; articula una relación clara entre problema, criterio dominante de calidad, arquitectura objetivo, recorte del MVP y prototipo observable. Esa articulación fortalece la coherencia argumentativa del manuscrito y permite defender por qué RAG, modularidad y escalamiento profesional no aparecen como accesorios, sino como decisiones necesarias para sostener trazabilidad clínica.

La validación funcional inicial muestra que el repositorio ya materializa el flujo principal del sistema mediante frontends diferenciados, una API con contratos mínimos, un motor de triaje y una recuperación documental que preserva evidencia. Aunque esta base aún no demuestra calidad clínica ni rendimiento operativo, sí deja una plataforma razonable para la siguiente fase del proyecto.

Referencias

- [1] J. Martinez-Selles et al. Concordancia diagnostica de herramientas de triaje digital en urgencias. *Emergencias*, 29(6):391–396, 2017.
- [2] Neha F. et al. Retrieval-augmented generation (rag) in healthcare: A comprehensive review. *AI Journal*, 2024.
- [3] World Health Organization. Digital health enterprise architecture (dhea), 2025. Documento de referencia.
- [4] Amazon Web Services. Soluciones en la nube para proveedores de salud, 2024. Documento tecnico.

- [5] Y. Shen et al. Performance of emergency triage prediction using chatgpt. *PMC*, 2023.
- [6] D. K. Rodriguez. Interoperabilidad y uso de datos clinicos en tiempo real en colombia. Portafolio, 2026.
- [7] Ministerio de Ciencia, Tecnologia e Innovacion. Marco etico para la inteligencia artificial en colombia, 2021.

Cuadro 4: Resumen ADR

Decisión	Alternativas	Justificación	Impacto
FastAPI para backend	Spring Boot, NestJS	Reduce fricción para integrar IA, contratos REST y validación con Pydantic.	Facilita evolución rápida del MVP y prototipado del flujo RAG.
React para frontends	Angular, SSR simple	Permite dos experiencias separadas con componentes reutilizables y bajo costo de entrada.	Mejora la claridad de canales y roles en el prototipo.
JWT por roles	Sesiones simples, proveedor externo	Mantiene una seguridad básica suficiente para separar paciente y profesional.	Soporta trazabilidad por identidad y escalamiento.
PostgreSQL + vectorial	Solo relacional, NoSQL + vectorial	Separa datos transaccionales de evidencia semántica con una pila conocida.	Simplifica la evolución del MVP hacia nube sin cambiar el modelo base.
RAG clínico	LLM sin grounding, reglas puras	Aumenta trazabilidad y reduce dependencia de respuestas no fundamentadas.	Refuerza el atributo de calidad dominante.
FHIR conceptual	Integración ad hoc, sin interoperabilidad	Permite alinear la propuesta con estándares de salud sin forzar una integración inexistente.	Prepara la próxima entrega sin sobreprometer alcance.

Cuadro 5: Contratos API minimos

Endpoint	Método	Responsabilidad
/auth/login	POST	Autenticar al usuario y devolver un token de sesión.
/consultations	POST	Crear una consulta con sintomas y contexto basal.
/consultations/{id}	GET	Consultar el estado completo de una consulta.
/consultations/{id}/triage	POST	Ejecutar el flujo de triage y recomendacion.
/consultations/{id}/recommendation	GET	Obtener la recomendacion y la evidencia asociada.
/professional/cases	GET	Listar casos escalados pendientes de revision.
/professional/cases/{id}	GET	Consultar detalle de un caso escalado.

Cuadro 6: Validacion funcional inicial del MVP

Req.	Evidencia observable	Interpretacion
RF-01	Ruta <code>/auth/login</code> y separacion de rol <code>patient/professional</code> .	Existe autenticacion academica suficiente para separar los dos canales del MVP.
RF-02	Vista de captura en frontend y endpoint <code>POST /consultations</code> .	La consulta puede crearse a partir de sintomas, motivo principal y notas de contexto.
RF-03	Normalizacion del caso en <code>ConsultationsCreate</code> -> <code>ConsultationRecord</code> .	La entrada libre se transforma en una estructura interna consistente.
RF-04	Motor de triage basado en intensidad y palabras clave de alerta.	El sistema produce severidad, decision y racional inicial.
RF-05	Servicio RAG que carga el corpus local y recupera hasta tres fuentes.	La recomendacion no depende solo de generacion libre, sino de evidencia documental recuperada.
RF-06	Respuesta con resumen, severidad, <i>disclaimer</i> y <i>evidence sources</i> .	La salida es trazable y explicable en el nivel academico del MVP.
RF-07	Creacion automatica de <code>EscalationCase</code> cuando la severidad es alta.	El escalamiento esta modelado como comportamiento de negocio y no solo como idea conceptual.
RF-08	Vista profesional y rutas <code>/professional/cases</code> .	El profesional puede revisar la cola de casos y la evidencia asociada.

Cuadro 7: Evaluacion cualitativa de atributos de calidad

Atributo	Evidencia actual	Lectura critica
Trazabilidad	La recomendacion conserva fuentes, fecha, severidad, decision y version de prompt.	Es el atributo mejor representado en el MVP y el principal diferenciador frente a respuestas no fundamentadas.
Seguridad basica	El backend restringe acceso por rol y evita que un paciente consulte casos ajenos.	La separacion es suficiente para el ambito academico, pero no equivale a seguridad productiva.
Modularidad	Autenticacion, consultas, triage, RAG y casos profesionales estan separados en rutas y servicios.	La estructura favorece evolucion posterior y pruebas por modulo, aunque aun no existe despliegue distribuido.
Usabilidad operativa	El frontend diferencia el espacio del paciente y el del profesional segun la tarea.	La interfaz facilita comunicar el flujo, pero todavia usa datos de demostracion y requiere validacion con usuarios.
Escalabilidad tecnica	La arquitectura objetivo contempla separacion cloud de canales, API y persistencias.	La escalabilidad esta defendida a nivel de diseño, no demostrada empiricamente en esta entrega.